

Deep Learning Assignment 1

Part A: Convolutional Neural Networks on CIFAR-100

Student: Pravin Patil

Framework: TensorFlow 2.19.0 / Keras

Dataset: CIFAR-100 (32×32 RGB, 100 classes)

Date: May 2026

Abstract

This report documents a systematic investigation of Convolutional Neural Networks (CNNs) applied to the CIFAR-100 image classification benchmark. Three experimental configurations are studied: (A1) a CNN trained from scratch, (A2) robustness evaluation under additive Gaussian noise, (A3) noise-augmented retraining to improve robustness, and (A4/A5) transfer learning via a frozen VGG16 backbone. Quantitative results are presented alongside a rigorous analysis of numerical methods, hyperparameter choices, and the accuracy–robustness trade-off inherent in each approach.

1 Introduction

Image classification at scale remains one of the central problems of computer vision. CIFAR-100 [1] provides a challenging 100-class benchmark with 60,000 colour images of size 32×32 pixels (50,000 training, 10,000 test). The low resolution forces a model to learn highly discriminative features from limited spatial information.

This assignment explores three model families:

1. A **custom CNN trained from scratch** with progressively increasing filter depth.
2. **Noise-augmented retraining** to improve robustness against Gaussian perturbations.
3. **Transfer learning** using VGG16 pre-trained on ImageNet as a frozen feature extractor.

The unifying theme is the *accuracy-robustness trade-off*: high accuracy on clean data does not imply resilience to distribution shift, and the three approaches occupy different positions along this spectrum.

2 Dataset and Pre-Processing

2.1 Data Statistics

Table 1: CIFAR-100 Dataset Summary

Split	Samples	Image Size	Channels	Classes
Training	50,000	32×32	3 (RGB)	100
Test	10,000	32×32	3 (RGB)	100

2.2 Normalisation

Raw pixel values $p \in \{0, \dots, 255\}$ are mapped to $[0, 1]$:

$$\hat{p} = \frac{p}{255}, \quad \hat{p} \in [0, 1]. \quad (1)$$

This affine rescaling ensures that the gradient magnitudes during back-propagation remain in a numerically stable range, preventing vanishing or exploding gradients in early layers.

2.3 Label Encoding

Integer class labels $y \in \{0, \dots, 99\}$ are converted to one-hot vectors $\mathbf{y} \in \{0, 1\}^{100}$ compatible with categorical cross-entropy loss. For label c :

$$y_k = \begin{cases} 1 & k = c \\ 0 & k \neq c \end{cases}, \quad k \in \{0, \dots, 99\}. \quad (2)$$

3 Model Architectures

3.1 A1 – Custom CNN from Scratch

The baseline model follows a classic *VGG-style* deepening strategy: the number of convolutional filters doubles at each block while spatial resolution is halved by max-pooling.

Table 2: Custom CNN Layer Specification

Layer	Type	Output Shape	Parameters
Input	–	$32 \times 32 \times 3$	0
Conv2D-1	Conv + ReLU	$30 \times 30 \times 32$	896
MaxPool-1	MaxPool 2×2	$15 \times 15 \times 32$	0
Conv2D-2	Conv + ReLU	$13 \times 13 \times 64$	18,496
MaxPool-2	MaxPool 2×2	$6 \times 6 \times 64$	0
Conv2D-3	Conv + ReLU	$4 \times 4 \times 128$	73,856
MaxPool-3	MaxPool 2×2	$2 \times 2 \times 128$	0
Flatten	–	512	0
Dense-1	FC + ReLU	128	65,664
Dropout(0.5)	Regulariser	128	0
Dense-2	FC + Softmax	100	12,900
Total			171,812

Design rationale. Three convolutional blocks with filter counts $\{32, 64, 128\}$ extract low- to high-level features hierarchically. Each 3×3 kernel preserves local receptive field efficiency. Dropout ($p = 0.5$) on the penultimate layer acts as an ensemble method during training.

3.2 A3 – Noise-Augmented CNN

The architecture is identical to A1. The key modification is in the training data: 20% of training images are replaced by noise-corrupted copies, creating an augmented training set of 60,000 images.

3.3 A4 – Transfer Learning with VGG16

VGG16 pre-trained on ImageNet is used as a frozen feature extractor. Because CIFAR-100 images are 32×32 – far smaller than VGG16’s typical 224×224 input – each image is first up-sampled by a factor of 3 to 96×96 via bilinear interpolation (UpSampling2D). A small classifier head ($256 \rightarrow 100$) is trained on top of the frozen convolutional features.

Table 3: VGG16 Transfer Learning Head

Layer	Type	Trainable Parameters
UpSampling2D	$3 \times$ bilinear	0
VGG16 base	Frozen conv blocks	0
Flatten	–	0
Dense(256)	FC + ReLU	2,097,408
Dropout(0.5)	Regulariser	0
Dense(100)	FC + Softmax	25,700
Total trainable		2,123,108

4 Numerical Methods

4.1 Loss Function

All models minimise **categorical cross-entropy**:

$$\mathcal{L}(\boldsymbol{\theta}) = -\frac{1}{N} \sum_{i=1}^N \sum_{k=0}^{99} y_{i,k} \log \hat{y}_{i,k}, \quad (3)$$

where $\hat{y}_{i,k} = \text{softmax}(\mathbf{z})_k$ is the predicted probability for class k of sample i . The log ensures the loss surface is bounded below and has well-defined gradients even near extreme predictions.

4.2 Optimisation Algorithm – Adam

All models use the **Adam** optimiser [2], which adapts per-parameter learning rates:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad (4)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2, \quad (5)$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}, \quad (6)$$

$$\theta_t = \theta_{t-1} - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}, \quad (7)$$

where default hyperparameters $\alpha = 10^{-3}$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-7}$ are used. Adam’s bias-correction terms $(1 - \beta_1^t, 1 - \beta_2^t)$ are critical in early epochs when the moment estimates are cold-started at zero.

4.3 Activation Functions

ReLU ($\max(0, x)$) is used in all hidden layers. It avoids the vanishing-gradient problem common with sigmoid/tanh activations in deep networks and is computationally trivial to differentiate. The **Softmax** output converts logits $\mathbf{z}$ to a valid probability distribution:

$$\text{softmax}(\mathbf{z})_k = \frac{e^{z_k}}{\sum_{j=0}^{99} e^{z_j}}. \quad (8)$$

4.4 Convolution Operation

For a 2D convolutional layer with kernel $\mathbf{W}$ of size $k \times k$ and F filters applied to input feature map $\mathbf{X}$:

$$\mathbf{Y}_{f,i,j} = \sum_c \sum_{m=0}^{k-1} \sum_{n=0}^{k-1} \mathbf{W}_{f,c,m,n} \mathbf{X}_{c,i+m,j+n} + b_f. \quad (9)$$

All 3×3 kernels with unit stride (no padding) are used; each pooling layer halves spatial dimensions, preserving the most dominant activations.

4.5 Gaussian Noise Model (A2)

Additive Gaussian noise with variance $\sigma^2 = 0.05$ is applied:

$$\tilde{x} = x + \epsilon, \quad \epsilon \sim \mathcal{N}(0, \sigma^2), \quad \sigma = \sqrt{0.05} \approx 0.2236, \quad (10)$$

followed by clipping to $[0, 1]$. Given that $\hat{p} \in [0, 1]$, the signal-to-noise ratio (SNR) is:

$$\text{SNR} = \frac{\mathbb{E}[\hat{p}^2]}{\sigma^2} \approx \frac{0.25}{0.05} = 5, \quad (11)$$

which represents a moderate perturbation – enough to significantly degrade a model that has not been exposed to noise during training.

5 Training Configuration and Hyperparameters

Table 4: Hyperparameter Summary Across All Models

Hyperparameter	A1 (Scratch)	A3 (Aug.)	A4 (VGG16)
Epochs	80	80	80
Batch size	128	128	64
Optimiser	Adam	Adam	Adam
Learning rate	10^{-3}	10^{-3}	10^{-3}
Dropout rate	0.5	0.5	0.5
Training samples	50,000	60,000	50,000
Data augmentation	None	Gaussian noise (20%)	Rotation, shifts, flip

Batch size choice. A batch size of 128 represents a compromise between gradient-estimate variance (smaller batches = noisier but regularising) and GPU throughput (larger batches = faster). VGG16 experiments use 64 due to the larger intermediate feature maps from the upsampled 96×96 inputs.

Number of epochs. Training for 80 epochs allows sufficient convergence of the A1 baseline without early stopping. Both A3 and A4 use the same epoch budget for a fair comparison.

6 Results and Analysis

6.1 Quantitative Performance

Table 5: Model Performance on CIFAR-100 – Clean vs. Noisy Test Data

Model	Clean Acc.	Noisy Acc.	Abs. Drop	Rel. Drop
Scratch CNN (A1)	0.3825	0.0611	0.3214	84.07%
Noise-Augmented CNN (A3)	0.3678	0.2440	0.1238	33.66%
VGG16 Transfer (A4/A5)	0.4217	0.0266	0.3951	93.69%

6.2 Observation – Clean Data

The VGG16 transfer model achieves the highest clean-data accuracy (42.17%), benefiting from deep hierarchical feature representations learned on 1.2M ImageNet images. The scratch CNN reaches 38.25% – a respectable result for a lightweight model on a 100-class problem. The noise-augmented model (A3) trades a small 1.47 percentage point reduction in clean accuracy for drastically improved robustness.

6.3 Observation – Noisy Data

Both A1 and A4 collapse under Gaussian noise. The scratch CNN drops to 6.11% – near random chance for 100 classes (expected random: 1%). VGG16 fares even worse at 2.66%, suggesting the ImageNet features are tuned to high-frequency texture patterns that are catastrophically disrupted by $\sigma = 0.224$ noise. The noise-augmented A3 model maintains 24.40% accuracy, validating the strategy.

7 Trade-Off Analysis

7.1 Accuracy vs. Robustness Trade-Off

The core tension in this experiment is between **clean-data accuracy** and **noise robustness**. This is quantified by the *robustness ratio*:

$$\rho = \frac{\text{Acc}_{\text{noisy}}}{\text{Acc}_{\text{clean}}}. \quad (12)$$

Table 6: Robustness Ratio Comparison

Model	Clean Acc.	Robustness Ratio ρ
Scratch CNN (A1)	0.3825	0.160
Noise-Augmented CNN (A3)	0.3678	0.663
VGG16 Transfer (A4)	0.4217	0.063

A3 achieves a ρ of 0.663, more than $4\times$ better than the scratch CNN and $10\times$ better than VGG16. This is obtained at a cost of only ≈ 1.5 percentage points in clean accuracy.

7.2 Model Complexity vs. Accuracy Trade-Off

Table 7: Model Complexity vs. Performance

Model	Parameters	Training Data	Clean Acc.
Scratch CNN (A1/A3)	171,812	50,000 / 60,000	0.3825 / 0.3678
VGG16 Transfer (A4)	2,123,108 trainable	50,000	0.4217
VGG16 total params	$\approx 138\text{M}$ (frozen)	–	–

VGG16 obtains +3.92 pp over A1 by leveraging 138M pretrained parameters, but its trainable head alone contains $12.4\times$ more parameters than the entire scratch CNN. For embedded or edge deployments, A3 offers a better parameter-efficiency vs. robustness profile.

7.3 Computational Cost Trade-Off

Table 8: Qualitative Computational Cost Comparison

Model	Training Time	Inference Cost	Memory
Scratch CNN (A1)	Low	Low	Low
Noise-Augmented CNN (A3)	Medium	Low	Low
VGG16 Transfer (A4)	Medium	High	High

The VGG16 approach requires upsampling every 32×32 image to 96×96 at inference time, tripling the spatial footprint and significantly increasing intermediate activation memory. The scratch models process the native resolution, making them $3\text{--}5\times$ faster at inference.

7.4 Data Augmentation Trade-Off (A3)

Adding 20% noisy copies increases training set size from 50,000 to 60,000 (a 20% increase). The data augmentation introduces an inductive bias: the model learns that pixel-level Gaussian perturbations should not change the class label. This biases the decision boundary to be smooth in the ℓ_2 -ball around training points:

$$f(\mathbf{x} + \boldsymbol{\epsilon}) \approx f(\mathbf{x}), \quad \|\boldsymbol{\epsilon}\|_2 \leq r, \quad (13)$$

which is consistent with Lipschitz continuity of the learned function. The slight clean-accuracy drop ($0.3825 \rightarrow 0.3678$) reflects the model allocating representational capacity to the noise manifold at the cost of fitting fine-grained clean-image features.

8 Discussion

Why does VGG16 fail under noise? ImageNet features learned by VGG16 encode high-frequency texture and colour statistics. Gaussian noise with $\sigma = 0.224$ is spectrally

broad and heavily corrupts these high-frequency components, making the frozen feature representations unreliable for downstream classification. Furthermore, the upsampling step (UpSampling2D) creates block artefacts that interact poorly with noise, compounding the distribution shift.

Why does A3 succeed? Noise-augmented training is a form of *data augmentation regularisation*. It implicitly trains the model to learn class-relevant low-frequency features that persist under additive noise, rather than over-fitting to texture patterns. This is related to adversarial training: exposing the model to perturbed inputs during training improves robustness to the same perturbation at test time.

Limitations. The improved robustness of A3 is noise-type-specific. A model trained against Gaussian noise may remain fragile to adversarial examples, blur, JPEG compression, or geometric distortions. A comprehensive robustness strategy would use diverse augmentation (random crops, colour jitter, CutMix, AugMix) alongside noise injection.

9 Conclusion

This report compared three CNN-based approaches to CIFAR-100 classification with a focus on robustness to additive Gaussian noise:

1. **Scratch CNN (A1)** achieves 38.25% accuracy on clean data but degrades to 6.11% under noise – an 84% relative drop.
2. **Noise-Augmented CNN (A3)** reduces clean accuracy slightly to 36.78% but maintains 24.40% under noise, yielding a robustness ratio $4\times$ better than A1.
3. **VGG16 Transfer Learning (A4)** achieves the best clean-data accuracy (42.17%) but is the most fragile under noise (2.66%), performing near random-chance with a 93.7% relative drop.

The central finding is that **accuracy on clean data and robustness to noise are not automatically coupled**. Transfer learning from ImageNet – despite its accuracy advantage – introduces a strong prior over high-frequency features that makes it particularly sensitive to pixel-level perturbations. Noise-augmented training provides the best accuracy–robustness trade-off among the three strategies, at minimal additional computational cost.

Future work should explore Batch Normalisation in the scratch architecture, fine-tuning of the VGG16 backbone with noise-augmented data, and the use of certified defences such as randomised smoothing.

9 References

- [1] A. Krizhevsky, *Learning Multiple Layers of Features from Tiny Images*, Technical Report, University of Toronto, 2009.
- [2] D. P. Kingma and J. Ba, *Adam: A Method for Stochastic Optimization*, arXiv:1412.6980, 2014.
- [3] K. Simonyan and A. Zisserman, *Very Deep Convolutional Networks for Large-Scale Image Recognition*, ICLR 2015.

- [4] N. Srivastava et al., *Dropout: A Simple Way to Prevent Neural Networks from Overfitting*, JMLR, 15(1):1929–1958, 2014.
- [5] V. Nair and G. E. Hinton, *Rectified Linear Units Improve Restricted Boltzmann Machines*, ICML 2010.